Reservoir, Analog, and Reversible Computing

Timothy Li

June 2026

1 Reservoir Computing

1.1 Overview

Reservoir computing is a machine-learning method for working with data that changes over time. A reservoir computer has an input layer, a reservoir, and an output layer. The reservoir is usually a fixed network with nonlinear dynamics. Inputs are sent into this network, which turns them into a larger set of reservoir states. The output layer then uses these states to make a prediction or classification.

The main difference from a normal recurrent neural network is that the connections inside the reservoir are not trained. Only the output layer, also called the readout, is trained. The readout is commonly fitted using linear or ridge regression. This makes training much simpler because backpropagation does not have to be applied to all the recurrent connections.

The basic reservoir update can be written as

$$\mathbf{X}(t_i) = \mathbf{f}(\mathbf{X}(t_{i-1}), \mathbf{u}(t_i)) \,, \tag{1}$$

where $\mathbf{X}(t_i)$ is the reservoir state, $\mathbf{u}(t_i)$ is the current input, and $\mathbf{f}$ describes how the reservoir changes. In a physical reservoir, it may not be possible to measure every part of $\mathbf{X}(t)$. The part that can be measured is written as $\mathbf{x}(t)$. A simple readout is

$$\mathbf{y}(t) = \mathbf{W}_{\text{out}}\mathbf{x}(t) + \mathbf{b}_{\text{out}}, \tag{2}$$

where $\mathbf{W}_{\text{out}}$ and $\mathbf{b}_{\text{out}}$ are the trained weights and bias.

Echo state networks are a common digital version of reservoir computing. Reservoirs can also be made from physical systems such as analog circuits, photonic systems, magnetic devices, mechanical systems, and even fluids. [1] Inputs can be represented by voltages, forces, positions, pulses, or waves, depending on the system. The response is measured and used as the reservoir state.

If a system has only one physical node, it can still produce several *virtual nodes* through time multiplexing. This means measuring the same node at several points in time and treating those measurements as different reservoir states. [2] Reservoirs also have short-term, or fading, memory because their current state depends partly on earlier inputs.

1.2 Killer Problem

Reservoir computing is useful for problems involving time and memory. Examples include time-series forecasting, signal classification, system identification, anomaly detection, and control. It can be much easier to train than a standard recurrent neural network because only the readout is trained.

Physical reservoir computing is especially interesting because a physical system can provide the nonlinear dynamics directly. This could reduce the cost of running a large recurrent network, especially if sensing and computing happen in the same device. However, whether it is actually faster or more energy efficient depends on the full system, including how data enter and leave the reservoir.

[1] https://doi.org/10.1007/s11047-024-09997-y

[2] https://doi.org/10.1038/ncomms1476

1.3 Simulators

I looked at ReservoirPy, EchoTorch, and PRCpy. These tools do not all do the same thing.

ReservoirPy. ReservoirPy is the easiest tool I found for making a digital reservoir. It can create and train echo state networks using only a few lines of Python. It also includes tutorials, datasets, metrics, and tools for changing reservoir settings. This makes it a good starting point for testing reservoir computing and creating a digital baseline. [3]

ReservoirPy simulates a digital recurrent reservoir. It does not simulate the physics of a particular physical reservoir unless the user writes that model separately.

EchoTorch. EchoTorch is a PyTorch-based reservoir-computing library. It includes echo state networks, deep reservoirs, conceptors, datasets, and several research examples. I originally found it strange and difficult to work with, but the GitHub project has been updated since then. The current repository lists modern PyTorch versions and has testing workflows and recent contributor activity. [4]

EchoTorch should therefore not be described as deprecated. It is mainly a research tool, and its public PyPI release is still from 2018, so the packaged version does not fully represent the current GitHub code. [5] ReservoirPy may still be easier for a simple digital baseline, while EchoTorch may be useful when PyTorch integration or its more specialized reservoir models are needed.

PRCpy. PRCpy is meant for processing data from a physical reservoir. It can load experimental data, prepare it, train a readout, and evaluate the results. [6] It does not actually simulate the physical system. Normally, the user needs measurements from a real reservoir. Simulated data can also be used, but they must come from a separate simulator.

SciPy and other simulators. SciPy is not a reservoir-computing simulator by itself. However, its ODE solvers can simulate a dynamical system that is being used as a reservoir. The simulated positions, voltages, or other state variables can then be used as reservoir nodes. SciPy is also useful as a normal digital solver for comparing against analog or physical results.

For most physical reservoirs, there is no special reservoir simulator. Instead, a simulator for that physical system is used. For example, an electronic reservoir can use SPICE, a mechanical reservoir can use an ODE or finite-element solver, and an optical or magnetic reservoir needs a simulator for those systems. The results are then passed into an RC readout.

1.4 Disadvantages

Reservoir computing still requires choosing settings such as reservoir size, connection strength, and memory. A poor choice can make the reservoir unstable or unable to remember enough information.

Physical reservoirs also have noise, drift, limited numbers of measurable nodes, and differences between devices. Digital-to-analog and analog-to-digital conversion may be needed to send inputs into the reservoir and read its states. These parts can use a large amount of the total energy. A simulation may also look good even when the real system is difficult to build or measure.

1.5 Conclusion

Reservoir computing is a useful connection between machine learning and dynamical systems. ReservoirPy is a simple way to build a digital reservoir. EchoTorch is a more research-focused PyTorch option with some specialized models, while PRCpy is mainly useful after physical data already exist. For a new physical reservoir, the normal approach is to simulate the physical system with SciPy or another system-specific simulator and then train a simple readout on its states.

[3] `https://reservoirpy.readthedocs.io/en/stable/user_guide/quickstart.html`
[4] `https://github.com/nschaetti/EchoTorch`
[5] `https://pypi.org/project/EchoTorch/`
[6] `https://github.com/OSJL-py/PRCpy`

2 Analog Computing

2.1 Overview

Analog computing uses continuous physical quantities, such as voltages, to represent variables. In an electronic analog computer, a circuit is wired so that its voltages follow the same equations as the problem being studied. Initial conditions and parameters are set, and the output voltages show how the solution changes over time.

Operational amplifiers can perform operations such as addition, subtraction, integration, and differentiation. [7] Multipliers and other nonlinear elements can be added when a problem is nonlinear. This makes it possible to build circuits that represent many differential equations.

Analog computing is a larger field that also includes mechanical, optical, and other continuous systems. Here I am mainly discussing electronic analog computers made from circuit components.

2.2 Killer Problem

Analog computers are especially useful for modeling dynamical systems. Ordinary differential equations can be built directly from integrators, adders, and multipliers. Partial differential equations can also be modeled after they are broken into a set of ordinary differential equations, although a large problem may need many circuit elements.

The circuit does not solve the equation "instantly," but all of its parts operate at the same time. This can make it useful for real-time simulation and control. Analog computing can also perform linear algebra, but large matrix operations may require too many components and connections. [8]

2.3 Simulators

I looked at ngspice and LTspice, PyAnalog, and lucipy. They work at different levels.

ngspice and LTspice. ngspice and LTspice are SPICE circuit simulators. The user builds a circuit using a netlist or schematic, and the simulator calculates voltages and currents. These are the best tools here for checking a real electrical design, including component values, bandwidth, saturation, and device models. [9] [10]

SPICE does not automatically turn a differential equation into an analog-computer circuit. The circuit must be designed first. It is also important to remember that SPICE is still a digital simulation, so its speed does not show the speed of real analog hardware.

PyAnalog. PyAnalog works at a higher level. It represents an analog computer using elements such as integrators, summers, and multipliers. It can simulate these abstract circuits and can help translate a differential equation into a form that could be placed on an analog computer. [11]

PyAnalog is helpful for checking the mathematical circuit, but it is not as detailed as SPICE. A reasonable workflow is to use PyAnalog for the equation-level design and SPICE for the final electrical circuit.

lucipy. lucipy was made for Anabrid's LUCIDAC hybrid analog computer. It can configure the hardware, run programs, and collect data. It also includes an idealized circuit simulator and a LUCIDAC emulator. [12]

The circuit simulator solves the mathematics of a LUCIDAC circuit. The emulator imitates the LUCIDAC device interface so that client code can be tested without the machine. Neither one fully represents the noise

[7] https://chem.libretexts.org/Bookshelves/Analytical_Chemistry/Instrumental_Analysis_(LibreTexts)/03:_Operational_Amplifiers_in_Chemical_Instrumentation_(TBD)/3.04:_Application_of_Operational_Amplifiers_to_Mathematical_Operations

[8] https://yipenghuang.com/wp-content/uploads/2018/09/mmi2017030030.pdf

[9] https://ngspice.sourceforge.io/index.html

[10] https://www.analog.com/en/resources/design-tools-and-calculators/ltspice-simulator.html

[11] https://anabrid.dev/docs/pyanalog/html/index.html

[12] https://anabrid.dev/docs/lucipy/dirhtml/

and other imperfections of real hardware. The lucipy repository was archived in 2026, so it may be better for existing examples than for a new long-term project. [13]

You do not need to buy a LUCIDAC to install lucipy or use its built-in simulator and emulator. Anabrid provides software-only examples that run without hardware. [14] However, to run the circuit on a real analog computer or collect real measurements, you need access to a LUCIDAC. This could mean buying one from Anabrid or using one owned by a laboratory. The emulator also has limitations, and some examples involving data acquisition still require a connection to the physical device.

2.4 Disadvantages

The biggest issue is converting between digital and analog data. DACs are needed to turn digital inputs into voltages, and ADCs are needed to measure the outputs digitally. These conversions can use enough time and energy to remove much of the benefit of analog computing.

Analog systems also have noise, component tolerances, drift, limited precision, and voltage limits. Large problems need more physical computing elements and more connections, which makes scaling difficult.

2.5 Conclusion

Analog computing is promising for real-time dynamical systems. PyAnalog and lucipy can help check the mathematical setup, while SPICE can test a more realistic circuit. However, a software simulation cannot completely show the accuracy, speed, or energy use of actual analog hardware.

3 Reversible Computing

3.1 Overview

Reversible computing is based on the idea that information should not be erased during a calculation. Landauer's principle says that erasing one bit has a minimum energy cost of

$$E_{\min} = k_{\mathrm{B}} T \ln 2,$$

where k_{B} is Boltzmann's constant and T is temperature. Reversible logic tries to avoid this cost by using one-to-one operations, meaning that the input can always be recovered from the output.

Normal AND and OR gates are not reversible because several inputs can produce the same output. Reversible circuits instead use gates such as CNOT, Toffoli, Fredkin, and Peres gates. For example, a CNOT gate keeps A unchanged and flips B when $A = 1$:

Input		Output	
A	B	P	Q
0	0	0	0
0	1	0	1
1	0	1	1
1	1	1	0

The Toffoli gate flips a target bit when two control bits are both one. The Fredkin gate swaps two bits when its control bit is one. The Peres gate combines operations that are useful in reversible arithmetic. These gates can be combined to represent general logic.

[13] `https://github.com/anabrid/lucipy`

[14] `https://anabrid.dev/docs/lucipy/dirhtml/example-circuits/`

3.2 Killer Problem

Reversible computing does not have a special problem that normal digital computers cannot solve. Its main proposed benefit is saving energy by avoiding information loss. It is therefore more of a way to build computers than a new type of application.

Quantum computing uses reversible, unitary gates, so reversible logic is important for quantum circuits. However, classical reversible computing does not include superposition or entanglement, so it is not the same as quantum computing.

3.3 Simulators

I tested RevKit and MQT SyReC. Both worked, but they are mainly circuit-synthesis tools rather than physical hardware simulators.

RevKit. RevKit works closer to the logic and gate level. The input can describe a truth table, reversible function, or logic network. RevKit then creates or improves a reversible circuit and can test whether the circuit gives the correct outputs. [15]

MQT SyReC. MQT SyReC works at a higher level. The user writes a program in the SyReC hardware-description language, and the tool turns it into a reversible circuit. It includes options that trade the number of circuit lines against the number and cost of gates. The final circuit can also be simulated on normal bit inputs. [16]

RevKit is more useful when starting from a logic function, while MQT SyReC is more useful when starting from a higher-level program. Their simulators check whether the logic works. They do not calculate the electrical behavior or actual energy use of a physical reversible computer.

3.4 Disadvantages

Reversible circuits often need extra temporary bits and extra gates so that no information is destroyed. These temporary values must later be removed by reversing part of the calculation. This can make a reversible circuit much larger than a normal circuit.

Also, a logically reversible circuit does not automatically use almost no energy. The physical hardware must be designed to switch with very low loss. RevKit and MQT SyReC cannot show whether a real circuit will reach that goal.

3.5 Conclusion

Reversible computing is probably not the best direction if the goal is to find a unique killer problem. It is still useful for studying low-energy logic and quantum circuits. RevKit and MQT SyReC are good for designing and checking reversible logic, but a different type of simulator would be needed to study physical energy use.

4 Overall Assessment

Reservoir computing currently has the easiest path for software experiments because ReservoirPy provides a simple digital baseline, and many different dynamical systems can be tested as reservoirs. Analog computing is promising for real-time differential equations, but its advantages need to be tested with real hardware and data conversion included. Reversible computing is useful for energy-saving logic and quantum circuits, but it does not have a clear killer application of its own.

[15] https://msoeken.github.io/revkit.html
[16] https://mqt.readthedocs.io/projects/syrec/en/stable/index.html